

Projeto Stack

Participantes:

Pedro Arthur Santana Patriota

Yasmim Vitoria Silva de Oliveira

Pedro Augusto de Almada Lobo Fonseca

Arthur Brito Medeiros

1. Decisões de implementação (*design*)

Estrutura de Dados

pintos/src/threads/thread.h

```
...  
void thread_exit (void) NO_RETURN;  
void thread_exit_verbose (int exit_status) NO_RETURN;  
void thread_yield (void);  
...
```

thread_exit_verbose: Nova função declarada permitir que threads de usuário encerrem com um código de saída explícito.

```
void  
thread_exit_verbose (int exit_status)  
{  
    struct thread *cur_thread = thread_current ();  
    printf ("%s: exit(%d)\n", cur_thread->name, exit_status);  
    thread_exit ();  
}
```

Algoritmos

Funções Criadas

pintos/src/userprog/process.c

```
static char *  
extract_prog_name (const char *file_name)  
{  
    char *save_ptr;  
    return strtok_r ((char *) file_name, " ", &save_ptr);  
}
```

extract_prog_name: Função auxiliar que extrai apenas o nome do executável a partir da linha de comando completa, usando *strtok_r* para separar tudo antes do primeiro espaço

```

static void
validate_user_address (const uint8_t *uaddr)
{
    struct thread *cur = thread_current ();

    if (uaddr == NULL
        || !is_user_vaddr (uaddr)
        || cur->pagedir == NULL
        || pagedir_get_page (cur->pagedir, uaddr) == NULL)
        thread_exit_verbose (-1);
}

static uint8_t
get_user_byte (const uint8_t *uaddr)
{
    validate_user_address (uaddr);
    return *(uint8_t *) pagedir_get_page (thread_current ()->pagedir,
uaddr);
}

```

```

static uint32_t
get_user_word (const void *uaddr)
{
    const uint8_t *bytes = uaddr;
    uint32_t word = 0;
    size_t i;

    for (i = 0; i < sizeof (uint32_t); i++)
        word |= (uint32_t) get_user_byte (bytes + i) << (i * 8);
    return word;
}

```

get_user_word: Função auxiliar para ler valores de 32 bits a partir de endereços virtuais do usuário com segurança

Funções modificadas

pintos/src/userprog/process.c

```

tid_t process_execute (const char *file_name)
{
    ...
    char *prog_name = extract_prog_name (file_name);

    /* Create a new thread to execute FILE_NAME. */
    tid = thread_create (prog_name, PRI_DEFAULT, start_process, fn_copy);
    if (tid == TID_ERROR)

```

```

    palloc_free_page (fn_copy);
    return tid;
}

```

process_execute: Modificada para usar *extract_prog_name* ao criar a thread para que o nome exibido na thread seja o processo sendo executado e não a linha de comando inteira. A versão original de **process_execute()** chamava **strtok_r()** diretamente sobre a string recebida para extrair o nome do programa. Isso alterava a string antes de ela ser repassada para **start_process()**, então a linha de comando chegava incompleta na hora de montar a *stack*.

```

int process_wait (tid_t child_tid UNUSED)
{
    /* Temporary implementation: busy-wait so the parent doesn't
       exit before the child finishes. Will be replaced with
       proper synchronization in Project 2-2 (syscalls). */
    volatile int i;
    for (i = 0; i < 300000000; i++)
        ;
    return -1;
}

```

process_wait: Ainda usa um busy wait provisório — um loop vazio que roda 300 milhões de iterações para dar tempo ao processo filho de executar. Funciona para os testes atuais porque o filho termina antes do loop acabar, mas não serve para o Projeto 2 completo: sem sincronização real entre pai e filho, o pai não sabe quando o filho terminou nem consegue recuperar o exit status dele.

```

bool load (const char *file_name, void (**eip) (void), void **esp)
{
    struct thread *t = thread_current ();
    struct Elf32_Ehdr ehdr;
    @@ -215,18 +230,25 @@ load (const char *file_name, void (**eip) (void),
void **esp)
    bool success = false;
    int i;

    char *cmd_line_copy = palloc_get_page (0);
    if (cmd_line_copy == NULL)
        return false;
    strcpy (cmd_line_copy, file_name, PGSIZE);

    char *prog_name = extract_prog_name (file_name);

    ...

    /* Open executable file. */
    file = filesys_open (prog_name);
}

```

```

if (file == NULL)
{
    printf ("load: %s: open failed\n", prog_name);
    goto done;
}
...
/* Set up stack. */
if (!setup_stack (esp, cmd_line_copy))
    goto done;

/* Start address. */
*eip = (void (*) (void)) ehdr.e_entry;
success = true;

done:
/* We arrive here whether the load is successful or not. */
palloc_free_page (cmd_line_copy);
file_close (file);
return success;
}

```

load: Modificada para usar *extract_prog_name* para abrir corretamente o executável no sistema de arquivos e alocar uma cópia da linha de comando completa em uma página separada antes que *strtok_r* a modifique. Isso é necessário para que *setup_stack* tenha os argumentos completos na montagem da pilha.

```

setup_stack (void **esp, const char *cmd_line)
{
    uint8_t *kpage;
    bool success = false;

    kpage = palloc_get_page (PAL_USER | PAL_ZERO);
    if (kpage != NULL)
    {
        success = install_page (((uint8_t *) PHYS_BASE) - PGSIZE, kpage, true);
        if (success)
        {
            *esp = PHYS_BASE;

            /* Tokenize the command line and collect arguments. */
            char *token, *save_ptr;
            int argc = 0;
            char *argv[128];
            char *cmd_copy = palloc_get_page (0);
            if (cmd_copy == NULL)
                return false;
            strcpy (cmd_copy, cmd_line, PGSIZE);

            for (token = strtok_r (cmd_copy, " ", &save_ptr); token != NULL;
                token = strtok_r (NULL, " ", &save_ptr))

```

```

        argv[argc++] = token;

        /* Push argument strings onto the stack (right to left). */
        int i;
        char *argv_addrs[128];
        for (i = argc - 1; i >= 0; i--)
        {
            size_t len = strlen (argv[i]) + 1;
            *esp -= len;
            memcpy (*esp, argv[i], len);
            argv_addrs[i] = *esp;
        }

        /* Word-align. */
        *esp = (void *)((uintptr_t) *esp & ~3);

        /* Push null sentinel (argv[argc]). */
        *esp -= sizeof (char *);
        *(char **)*esp = NULL;

        /* Push argv[i] pointers (right to left). */
        for (i = argc - 1; i >= 0; i--)
        {
            *esp -= sizeof (char *);
            *(char **)*esp = argv_addrs[i];
        }

        /* Push argv (pointer to argv[0]). */
        char **argv_start = *esp;
        *esp -= sizeof (char **);
        *(char ***)*esp = argv_start;

        /* Push argc. */
        *esp -= sizeof (int);
        *(int *)*esp = argc;

        /* Push fake return address. */
        *esp -= sizeof (void *);
        *(void **)*esp = NULL;

#ifdef 0
        /* Print the stack. Make sure to keep this DISABLED during tests. */
        hex_dump ((uintptr_t) *esp, *esp,
                  (uintptr_t) PHYS_BASE - (uintptr_t) *esp, true);
#endif

        palloc_free_page (cmd_copy);
    }
    else
        palloc_free_page (kpage);
}

```

setup_stack: Modificada para implementar a convenção x86, seguindo o padrão de pilha do usuário exigido pelo Pintos. Agora, recebemos o *cmd_line*, além do ponteiro da pilha. Com o *cmd_line*, o partimos em partes de argumentos individuais e as empilhamos do último ao primeiro. Depois, alinhamos os endereços em múltiplos de 4 e preenchemos o offset com endereço nulo. Depois disso, preenchemos *argv[argc]* como nulo e iniciamos o preenchimento de *argv[i]* com os endereços dos argumentos. Por fim, preenchemos *argv*, *argc* e um endereço nulo

Para ilustrar, considerando a linha de comando `"/bin/ls -l foo bar"`, a stack fica assim ao final da montagem:

Endereço	Conteúdo	Descrição
0xbfffffc	"bar\0"	argv[3] string
0xbffff8	"foo\0"	argv[2] string
0xbffff5	"-l\0"	argv[1] string
0xbffffed	"/bin/ls\0"	argv[0] string
0xbffffec	(alinhamento)	word-align
0xbffffe8	0x00000000	argv[4] = NULL (sentinela)
0xbffffe4	0xbfffffc	argv[3] → "bar"
0xbffffe0	0xbffff8	argv[2] → "foo"
0xbffffdc	0xbffff5	argv[1] → "-l"
0xbffffd8	0xbffffed	argv[0] → "/bin/ls"
0xbffffd4	0xbffffd8	argv (ponteiro para argv[0])
0xbffffd0	0x00000004	argc = 4
0xbffffcc	0x00000000	endereço de retorno falso

```
static void
syscall_handler (struct intr_frame *f)
{
    uint32_t *esp = (uint32_t *)f->esp;
    int syscall_num = get_user_word (esp);

    switch (syscall_num)
```

```

{
    case SYS_HALT:
        shutdown_power_off ();
        NOT_REACHED ();

    case SYS_EXIT:
    {
        int status = (int)get_user_word (esp + 1);
        f->eax = status;
        thread_exit_verbose (status);
        NOT_REACHED ();
    }

    case SYS_WRITE:
    {
        int fd = (int)get_user_word (esp + 1);
        const char *buff = (const char *)get_user_word (esp + 2);
        /* Since get_user_word returns a 32-bit value, we cast it to
           unsigned int instead of size_t. */
        unsigned int size = (unsigned int)get_user_word (esp + 3);

        if (fd == 1)
        {
            putbuf (buff, size);
            f->eax = size;
        }
        else
            f->eax = -1;

        break;
    }

    default:
        thread_exit_verbose (-1);
        break;
}
}

```

syscall_handler: Modificada para lidar com o número do syscall vindo do topo da pilha de interrupção (esp). Passamos a tratar SYS_HALT (Desliga o sistema imediatamente), SYS_EXIT (Encerra o processo atual com o status informado) e SYS_WRITE (lê o descritor de arquivo, o buffer e o tamanho da pilha e escreve no console)

2. Testes

```
1 pintos -v -k -T 60 -qemu --filesystem-size=2 -p tests/userprog/args-none -a args-none -- -q -f run args-none < /dev/null 2> tests/userprog/args-none.errors > tests/userprog/args-none.output
2 perl -I./...../tests/userprog/args-none.ck tests/userprog/args-none tests/userprog/args-none.result
3 pass tests/userprog/args-none
4 pintos -v -k -T 60 -qemu --filesystem-size=2 -p tests/userprog/args-single -a args-single -- -q -f run 'args: single onearg' < /dev/null 2> tests/userprog/args-single.errors > tests/userprog/args-single.output
5 perl -I./...../tests/userprog/args-single.ck tests/userprog/args-single tests/userprog/args-single.result
6 pass tests/userprog/args-single
7 pintos -v -k -T 60 -qemu --filesystem-size=2 -p tests/userprog/args-multiple -a args-multiple -- -q -f run 'args: multiple some arguments for you!' < /dev/null 2> tests/userprog/args-multiple.errors > tests/userprog/args-multiple.output
8 perl -I./...../tests/userprog/args-multiple.ck tests/userprog/args-multiple tests/userprog/args-multiple.result
9 pass tests/userprog/args-multiple
10 pintos -v -k -T 60 -qemu --filesystem-size=2 -p tests/userprog/args-many -a args-many -- -q -f run 'args: many a b c d e f g h i j k l m n o p q r s t u v' < /dev/null 2> tests/userprog/args-many.errors > tests/userprog/args-many.output
11 perl -I./...../tests/userprog/args-many.ck tests/userprog/args-many tests/userprog/args-many.result
12 pass tests/userprog/args-many
13 pintos -v -k -T 60 -qemu --filesystem-size=2 -p tests/userprog/args-dbl-space -a args-dbl-space -- -q -f run 'args: dbl-space two spaces' < /dev/null 2> tests/userprog/args-dbl-space.errors > tests/userprog/args-dbl-space.output
14 perl -I./...../tests/userprog/args-dbl-space.ck tests/userprog/args-dbl-space tests/userprog/args-dbl-space.result
15 pass tests/userprog/args-dbl-space
16 pintos -v -k -T 60 -qemu --filesystem-size=2 -p tests/userprog/sc-bad-sp -a sc-bad-sp -- -q -f run sc-bad-sp < /dev/null 2> tests/userprog/sc-bad-sp.errors > tests/userprog/sc-bad-sp.output
17 perl -I./...../tests/userprog/sc-bad-sp.ck tests/userprog/sc-bad-sp tests/userprog/sc-bad-sp.result
18 pass tests/userprog/sc-bad-sp
19 pintos -v -k -T 60 -qemu --filesystem-size=2 -p tests/userprog/sc-bad-arg -a sc-bad-arg -- -q -f run sc-bad-arg < /dev/null 2> tests/userprog/sc-bad-arg.errors > tests/userprog/sc-bad-arg.output
20 perl -I./...../tests/userprog/sc-bad-arg.ck tests/userprog/sc-bad-arg tests/userprog/sc-bad-arg.result
21 pass tests/userprog/sc-bad-arg
22 pintos -v -k -T 60 -qemu --filesystem-size=2 -p tests/userprog/sc-boundary -a sc-boundary -- -q -f run sc-boundary < /dev/null 2> tests/userprog/sc-boundary.errors > tests/userprog/sc-boundary.output
23 perl -I./...../tests/userprog/sc-boundary.ck tests/userprog/sc-boundary tests/userprog/sc-boundary.result
24 pass tests/userprog/sc-boundary
25 pintos -v -k -T 60 -qemu --filesystem-size=2 -p tests/userprog/sc-boundary-2 -a sc-boundary-2 -- -q -f run sc-boundary-2 < /dev/null 2> tests/userprog/sc-boundary-2.errors > tests/userprog/sc-boundary-2.output
26 perl -I./...../tests/userprog/sc-boundary-2.ck tests/userprog/sc-boundary-2 tests/userprog/sc-boundary-2.result
27 pass tests/userprog/sc-boundary-2
28 pintos -v -k -T 60 -qemu --filesystem-size=2 -p tests/userprog/sc-boundary-3 -a sc-boundary-3 -- -q -f run sc-boundary-3 < /dev/null 2> tests/userprog/sc-boundary-3.errors > tests/userprog/sc-boundary-3.output
29 perl -I./...../tests/userprog/sc-boundary-3.ck tests/userprog/sc-boundary-3 tests/userprog/sc-boundary-3.result
30 pass tests/userprog/sc-boundary-3
31 pintos -v -k -T 60 -qemu --filesystem-size=2 -p tests/userprog/halt -a halt -- -q -f run halt < /dev/null 2> tests/userprog/halt.errors > tests/userprog/halt.output
32 perl -I./...../tests/userprog/halt.ck tests/userprog/halt tests/userprog/halt.result
33 pass tests/userprog/halt
34 pintos -v -k -T 60 -qemu --filesystem-size=2 -p tests/userprog/exit -a exit -- -q -f run exit < /dev/null 2> tests/userprog/exit.errors > tests/userprog/exit.output
35 perl -I./...../tests/userprog/exit.ck tests/userprog/exit tests/userprog/exit.result
```

Os testes abaixo foram executados após os ajustes:

- tests/userprog/args-none: *PASS*
- tests/userprog/args-single: *PASS*
- tests/userprog/args-multiple: *PASS*
- tests/userprog/args-many: *PASS*
- tests/userprog/args-dbl-space: *PASS*
- tests/userprog/halt: *PASS*
- tests/userprog/exit: *PASS*
- tests/userprog/sc-bad-sp: *PASS*
- tests/userprog/sc-bad-arg: *PASS*
- tests/userprog/sc-boundary: *PASS*
- tests/userprog/sc-boundary-2: *PASS*
- tests/userprog/sc-boundary-3: *PASS*

Todos os 12 testes ligados à passagem de argumentos e à leitura segura da *stack* de *syscalls* passaram. Alguns testes fora do escopo desta entrega foram executados para medir o quanto falta:

- tests/userprog/create-normal: *FAIL*
- tests/userprog/open-normal: *FAIL*
- tests/userprog/wait-simple: *FAIL*

Essas falhas são esperadas: dependem de *syscalls* e sincronização que ainda não existem.

